

Chapter 4: LlamaIndex Local Storage & Persistence

Subfolder Reference Guide: [04-VectorDB/LlamaIndex/ & saved_index/](#)

1. Executive Summary & Storage Architecture

The **LlamaIndex** subfolder demonstrates how LlamaIndex handles **file-system persistence** for vector databases and document stores without requiring an external database server (like Milvus or Qdrant). By decoupling the in-memory index structure from storage using `StorageContext`, LlamaIndex serializes text nodes, vector embeddings, structural graphs, and index metadata directly into JSON files.

2. Subfolder File Inventory

File / Subfolder Path	Type / Format	Role & Description
.env.example	Config File	Environment variables specifying API keys (OpenAI / DeepSeek) and target storage path.
CreateLocalVectorStore-LlamaIndex.py	Python Script	Main script that builds <code>VectorStoreIndex</code> , persists it to disk via <code>StorageContext</code> , and demonstrates loading from disk without re-embedding.
saved_index/	Directory	Persisted storage directory created upon calling <code>index.storage_context.persist()</code> . Contains 5 distinct JSON store files.
saved_index/docstore.json	JSON Database	Document Store: Stores raw text chunks (Nodes), custom metadata, and parent-child node relationships.
saved_index/default__vector_store.json	JSON Database	Vector Store: Stores dense floating-point vector embeddings (e.g. 1536d / 1024d) mapped to Node IDs.
saved_index/index_store.json	JSON Database	Index Store: Stores index structural metadata (index ID, root nodes list, index type classification).
saved_index/image__vector_store.json	JSON Database	Multimodal Vector Store: Holds image embeddings for multimodal indices (empty/default if text-only).
saved_index/graph_store.json	JSON Database	Graph Store: Holds relation graphs and knowledge links between nodes (used for GraphRAG).

3. Deep-Dive: Understanding the 5 Persistence Files

When `index.storage_context.persist(persist_dir="./saved_index")` is invoked, LlamaIndex breaks the pipeline state into 5 discrete JSON files. Understanding each file's purpose is essential for debugging and deployment:

- **1. docstore.json (Document & Node Store)**

Holds the complete payload for every parsed chunk (Node). It contains text, metadata (e.g., file_name, page_number), and relationships (pointers to parent document ID, next node ID, previous node ID). When a query retrieves a node ID, LlamaIndex looks up the full text here to pass into the LLM prompt.

- **2. default__vector_store.json (Vector Store)**

Holds the numerical vector representations (e.g., 1536-dimensional arrays for OpenAI or 1024-dimensional arrays for BGE-M3). It maps node_id $\rightarrow$ embedding_vector. During retrieval, LlamaIndex computes cosine or dot-product similarity between the query vector and these stored arrays.

- **3. index_store.json (Index Structure Metadata)**

Acts as the master table of contents for the index. It records the index ID, the type of index (e.g., VectorStoreIndex or SummaryIndex), and a list of root node IDs. This allows LlamaIndex to know how to construct the query engine upon reloading.

- **4. image__vector_store.json (Multimodal Image Vector Store)**

Dedicated vector store for visual embeddings (e.g. CLIP / Visualized-BGE vectors). If your documents contain images and you use a multimodal index, image feature vectors are isolated here.

- **5. graph_store.json (Knowledge Graph & Structural Links)**

Stores subject-predicate-object triples and relational links if you build a Knowledge Graph index or explicit node hierarchy (e.g., Parent-Child / Small-to-Big retrieval).

4. Code Walkthrough: Save & Load Pattern

```
# 1. Building and Persisting the Index
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex, StorageContext
from llama_index.core import load_index_from_storage

documents = SimpleDirectoryReader("./data").load_data()
index = VectorStoreIndex.from_documents(documents)

# Persist all 5 JSON components to disk
index.storage_context.persist(persist_dir="./saved_index")

# 2. Reloading from Disk (Zero Embedding API Calls)
storage_context = StorageContext.from_defaults(persist_dir="./saved_index")
loaded_index = load_index_from_storage(storage_context)
query_engine = loaded_index.as_query_engine()
response = query_engine.query("What is the core setting of the game?")
```

5. Architectural Advantages of Local Index Storage

✓ **Cost & API Optimization:** Eliminates duplicate embedding API charges. Once a document corpus is processed and saved to saved_index/, subsequent application restarts load instantly from local disk.

✓ **Zero-Dependency Deployment:** Ideal for edge devices, desktop software, or offline environments where running a dedicated vector DB service (like Milvus or Qdrant) is unnecessary.

✓ **Modularity & Inspection:** Because the persistence format is human-readable JSON, developers can inspect docstore.json or index_store.json directly to debug chunking outputs or metadata fields.

